

ZERO-TO-JOB-READY DATA ANALYST 2026

M03 - SQL FOUNDATIONS

MySQL from zero | single-table reasoning | validate every query

QUESTION -> QUERY -> RESULT GRAIN -> VALIDATE -> EXPLAIN -> MASTER

Joins/CTEs/subqueries/windows are intentionally NOT taught in this module.

This book is your teacher

- Assume SQL is new. A database, schema, table, row grain, primary key, foreign key, NULL, query tab and result grid are explained before they are used as if you already know them.
- Type the SQL yourself. Copy/paste is allowed for the controlled lab-setup script only; practice queries should be typed so syntax becomes familiar.
- Every query has a question and an expected result grain. A query that runs is not automatically correct.
- Protected review opens only after a genuine saved attempt. Close the review, then solve a changed query before mastery.
- The setup script is the only controlled place where database-creation/reset commands are expected. Normal lesson practice is read-only SELECT reasoning.

Your SQL learning loop

1. Read the business question
2. Say expected result grain
3. Watch exact segment if assigned
4. Follow teacher query once
5. Type guided query
6. Close support + solve changed query
7. Validate count/value/grain
8. Explain in your own words

Contents / Index

SF0 Set up MySQL Workbench and run your first query	6
SF1 Relational basics, keys and NULL	9
SF2 SELECT, aliases and DISTINCT	12
SF3 WHERE and filter logic	15
SF4 ORDER BY and LIMIT	18
SF5 Aggregate functions	21
SF6 GROUP BY and result grain	24
SF7 HAVING vs WHERE	27
SF8 CASE for business classification	30
End-of-module mastery checklist	33
Revision / SQL quick reference	34

Module prerequisite and tool map

0. Prerequisite

Course progression reaches M03 after M02, but no SQL syntax is assumed. Foundation concepts such as row grain, validation and evidence are reused. Excel knowledge can help you recognize tables, but it is not used as a hidden SQL prerequisite.

Tools / files

- MySQL Community Server + MySQL Workbench. Use current official MySQL documentation/download pages if installer screens differ from the video.
- C1_M03_SQL_FOUNDATIONS_LAB_SETUP_RC1.sql - run the whole file once to create/reset the controlled course database.
- C1_M03_SQL_FOUNDATIONS_PRACTICE_TASKS_RC1.sql - type answers under each guided/fresh task.
- C1_M03_SQL_FOUNDATIONS_EVIDENCE_LOG_RC1.xlsx - append-only attempt/validation record.
- Protected Practice Review and Gate reviews - keep closed until the matching attempt is saved.

Module boundary

- Single-table SELECT/filter/sort/aggregate/GROUP BY/HAVING/CASE only.
- No JOIN task in M03.
- No CTE, subquery or window-function task in M03.
- Those are taught in M04 SQL for Analysts after this foundation is mastered.

Exact practice, review and Gate route

- SF0: install/connect, run the controlled setup script once, verify 12 customers and 48 orders, then run your first read-only SELECT.
- For every lesson: learner book -> mapped visual if useful -> teacher demo -> guided query in practice SQL -> changed fresh query -> validation -> explain-back -> Evidence_Log.
- Practice review is post-attempt only. Repair only the named weak clause/idea, close the review, and solve a changed query.
- After SF0-SF8, attempt Gate A. Gate work is single-table and uses a fresh domain.
- A clean Gate A pass ends the Gate. Gate B requires failed A + Review A + targeted repair + explicit assignment. Gate C requires failed B + Review B + targeted repair + explicit assignment.

Critical-failure examples

Wrong result grain, unsafe NULL logic, LIMIT without deterministic ordering when claiming top-N, incorrect aggregate/group logic, WHERE/HAVING confusion, CASE boundary failure or no validation can block a pass. No percentage-average shortcut.

SF0 - Set up MySQL Workbench and run your first query

Practice block: SF0 in C1_M03_SQL_FOUNDATIONS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

A SQL query can run and still answer the wrong question. This lesson builds one reliable piece of the single-table query pipeline and connects syntax to result grain and validation.

2. What you are learning

Open MySQL Workbench, connect safely, run the course setup script, verify 12 customer rows and 48 order rows, and run a read-only SELECT query without confusing the result grid with stored data.

3. What you already need / prerequisite

No SQL skill is assumed; follow SF0 setup carefully.

4. Picture it in your head

SQL is a request sent to a database server. Workbench is the client interface: connection -> schema -> query tab -> result grid. In this module you mainly read with SELECT; the provided setup script is the controlled exception that creates/reset course data.

5. New words before we use them

Server = database engine running in the background. Connection = saved route/credentials to the server. Schema/database = named container of tables. Query tab = where you type SQL. Result grid = rows returned by a query.

6. Watch / visual source

RECOMMENDED. Learn SQL Beginner to Advanced in Under 4 Hours - Alex The Analyst | 0:52-11:18. Why watch: See MySQL/Workbench setup visually, then use the current official Workbench docs/download if the interface differs from the video. Internal fallback: The learner book includes the complete SQL pattern, expected result logic and recovery path.

[Open mapped source](#)

SF0 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

Run `SELECT 1 AS connection_test`; then `USE da_sql_foundations`; `SELECT order_id, region, sales_amount FROM orders LIMIT 3`; Confirm that the result grid shows three rows and that the table itself was not modified.

8. What you should see / be able to say

Connection test returns 1; course setup returns 12 customers and 48 orders; the transfer query returns exactly 7 rows with the requested columns; no `UPDATE/DELETE/DROP/ALTER` is used outside the controlled setup script.

9. Why it worked

The query shape matches the question, the returned rows have an explicit grain, and an independent count/value/logic check can confirm the answer. SQL syntax is only useful when those three agree.

10. Common beginner mistakes

Trying to fix connection problems by changing SQL; running only part of the setup script and then querying a missing schema; confusing a result grid with stored table rows; executing destructive commands outside the controlled setup.

11. If you get stuck - recovery ladder

Unknown database -> rerun the full setup script. Connection error -> verify the MySQL service and credentials before changing SQL. Wrong result -> make sure the correct schema is active and only the intended statement is selected/run. 1) Restate the intended result grain/question. 2) Identify the relevant clause/function. 3) Reveal only a clause skeleton, not the final query. 4) Open the matching review only after a saved attempt. 5) Close review and solve a changed retry.

SF0 - Do alone, validate and master

12. Now you do a small version alone

Close the guided query. From a blank query tab, select order_id, order_date, category and sales_amount for 7 rows. Add a separate COUNT(*) query and explain why the count is a validation check rather than part of the 7-row answer.

13. Validate your result

Connection test returns 1; course setup returns 12 customers and 48 orders; the transfer query returns exactly 7 rows with the requested columns; no UPDATE/DELETE/DROP/ALTER is used outside the controlled setup script.

14. Teach it back in your own words

Explain server, connection, schema, query tab and result grid in beginner language. Then explain why running SELECT does not normally modify the stored table.

15. Protected review + changed fresh retry

Save the SF0 attempt in the Evidence Log first. If needed, open the protected SF0 review, diagnose only the weak idea, close the review, then solve a changed query from a blank query tab. Append the retry; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, a result screenshot/text export if required, at least one validation query/check, one error/recovery note, a one-line result-grain statement, and a concise business interpretation. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh changed retry. FAIL -> return only to this lesson's weak competency; do not restart the full SQL module.

SF1 - Relational basics, keys and NULL

Practice block: SF1 in C1_M03_SQL_FOUNDATIONS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

A SQL query can run and still answer the wrong question. This lesson builds one reliable piece of the single-table query pipeline and connects syntax to result grain and validation.

2. What you are learning

State the grain of customers and orders, distinguish primary and foreign keys, and filter missing values with IS NULL while explaining why NULL is not zero.

3. What you already need / prerequisite

Prerequisite: SF0. The new clause/idea is still taught from first principles.

4. Picture it in your head

Grain means what one row represents. customers is one row per customer; orders is one row per order. A primary key identifies a row at that grain. A foreign key can repeat on the many side. NULL means missing/unknown/not provided, so SQL uses IS NULL / IS NOT NULL rather than = NULL.

5. New words before we use them

Grain = what one row represents. Primary key = unique row identifier. Foreign key = value linking to another table's key. NULL = missing/unknown/not supplied.

6. Watch / visual source

BOOK_LED. Internal lesson + course SQL lab | . Why watch: Grain, primary/foreign key and NULL are taught against the exact course tables; no filler video is needed. Internal fallback: The learner book + lab/practice SQL are the canonical route.

SF1 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

SELECT order_id, customer_id, satisfaction_score FROM orders WHERE satisfaction_score IS NULL LIMIT 5; Notice that order_id should be unique, customer_id may repeat, and missing satisfaction is displayed as NULL rather than 0.

8. What you should see / be able to say

The NULL query contains only NULL scores. The NOT NULL query contains no NULL scores. COUNT(*) is greater than COUNT(satisfaction_score) on the course data. Your explanation says repeated customer_id values in orders can be legitimate because order grain is one row per order.

9. Why it worked

The query shape matches the question, the returned rows have an explicit grain, and an independent count/value/logic check can confirm the answer. SQL syntax is only useful when those three agree.

10. Common beginner mistakes

Calling repeated customer_id values duplicates even though order grain is one row per order; treating NULL as 0; writing = NULL instead of IS NULL; confusing primary and foreign keys.

11. If you get stuck - recovery ladder

If = NULL returns an unexpected/empty result, replace it with IS NULL. If you think repeated customer_id values are duplicates, restate the table grain before deleting or changing anything. 1) Restate the intended result grain/question. 2) Identify the relevant clause/function. 3) Reveal only a clause skeleton, not the final query. 4) Open the matching review only after a saved attempt. 5) Close review and solve a changed retry.

SF1 - Do alone, validate and master

12. Now you do a small version alone

Without the lesson open, write a query returning rows whose `satisfaction_score` is NOT NULL, then compare `COUNT(*)` with `COUNT(satisfaction_score)`. Explain what the difference tells you about missingness.

13. Validate your result

The NULL query contains only NULL scores. The NOT NULL query contains no NULL scores. `COUNT(*)` is greater than `COUNT(satisfaction_score)` on the course data. Your explanation says repeated `customer_id` values in orders can be legitimate because order grain is one row per order.

14. Teach it back in your own words

Explain grain, primary key, foreign key and NULL with one example from the course tables. State why a foreign key can repeat and why NULL is not the same as 0 or an empty string.

15. Protected review + changed fresh retry

Save the SF1 attempt in the Evidence Log first. If needed, open the protected SF1 review, diagnose only the weak idea, close the review, then solve a changed query from a blank query tab. Append the retry; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, a result screenshot/text export if required, at least one validation query/check, one error/recovery note, a one-line result-grain statement, and a concise business interpretation. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh changed retry. FAIL -> return only to this lesson's weak competency; do not restart the full SQL module.

SF2 - SELECT, aliases and DISTINCT

Practice block: SF2 in C1_M03_SQL_FOUNDATIONS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

A SQL query can run and still answer the wrong question. This lesson builds one reliable piece of the single-table query pipeline and connects syntax to result grain and validation.

2. What you are learning

Return only requested columns, use aliases for readable output, use DISTINCT correctly, and state the grain of a distinct combination.

3. What you already need / prerequisite

Prerequisite: SF1. The new clause/idea is still taught from first principles.

4. Picture it in your head

SELECT chooses the result columns. AS changes a result label, not the stored column. DISTINCT removes duplicate combinations across the selected columns; DISTINCT region, category produces one row per unique region + category pair.

5. New words before we use them

SELECT = choose result columns. Alias/AS = result label. DISTINCT = keep unique selected combinations. Result grain = what one returned row represents.

6. Watch / visual source

RECOMMENDED. Learn SQL Beginner to Advanced in Under 4 Hours - Alex The Analyst | 11:19-21:46. Why watch: See the exact SQL clause/interface behavior before typing the matching query yourself. Internal fallback: The learner book includes the complete SQL pattern, expected result logic and recovery path.

[Open mapped source](#)

SF2 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

SELECT DISTINCT region, category FROM orders ORDER BY region, category; Say the result grain aloud before interpreting the rows.

8. What you should see / be able to say

Alias labels appear only in the result. DISTINCT status + channel has no duplicate pairs. The result grain explanation matches the selected combination rather than the source-table grain.

9. Why it worked

The query shape matches the question, the returned rows have an explicit grain, and an independent count/value/logic check can confirm the answer. SQL syntax is only useful when those three agree.

10. Common beginner mistakes

Using DISTINCT without stating the new result grain; assuming AS renames the stored column; selecting extra columns that change which combinations are distinct.

11. If you get stuck - recovery ladder

If DISTINCT appears to remove too much/little, check exactly which columns are in SELECT. If aliasing causes confusion, remember that the stored table schema is unchanged by AS. 1) Restate the intended result grain/question. 2) Identify the relevant clause/function. 3) Reveal only a clause skeleton, not the final query. 4) Open the matching review only after a saved attempt. 5) Close review and solve a changed retry.

SF2 - Do alone, validate and master

12. Now you do a small version alone

Return unique status + channel combinations. Then return only order_id and sales_amount with readable aliases for completed orders. Explain why selecting fewer columns does not delete data from the table.

13. Validate your result

Alias labels appear only in the result. DISTINCT status + channel has no duplicate pairs. The result grain explanation matches the selected combination rather than the source-table grain.

14. Teach it back in your own words

Explain SELECT, AS and DISTINCT. Give one example where DISTINCT on one column and DISTINCT on two columns answer different questions.

15. Protected review + changed fresh retry

Save the SF2 attempt in the Evidence Log first. If needed, open the protected SF2 review, diagnose only the weak idea, close the review, then solve a changed query from a blank query tab. Append the retry; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, a result screenshot/text export if required, at least one validation query/check, one error/recovery note, a one-line result-grain statement, and a concise business interpretation. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh changed retry. FAIL -> return only to this lesson's weak competency; do not restart the full SQL module.

SF3 - WHERE and filter logic

Practice block: SF3 in C1_M03_SQL_FOUNDATIONS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

A SQL query can run and still answer the wrong question. This lesson builds one reliable piece of the single-table query pipeline and connects syntax to result grain and validation.

2. What you are learning

Translate business rules into reliable row filters using comparison operators, AND/OR with parentheses, BETWEEN, IN, LIKE and correct NULL logic.

3. What you already need / prerequisite

Prerequisite: SF2. The new clause/idea is still taught from first principles.

4. Picture it in your head

WHERE decides which source rows are eligible before any later grouping. Read the filter in plain language. Parentheses make mixed AND/OR rules explicit; BETWEEN is inclusive; IN expresses membership; LIKE expresses patterns; NULL uses IS NULL.

5. New words before we use them

WHERE = row filter before grouping. Predicate = TRUE/FALSE condition. AND = all conditions. OR = at least one. IN/BETWEEN/LIKE = common filter patterns.

6. Watch / visual source

RECOMMENDED. Learn SQL Beginner to Advanced in Under 4 Hours - Alex The Analyst | 21:47-33:53. Why watch: See the exact SQL clause/interface behavior before typing the matching query yourself. Internal fallback: The learner book includes the complete SQL pattern, expected result logic and recovery path.

[Open mapped source](#)

SF3 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

SELECT order_id, region, category, status, sales_amount FROM orders WHERE status='Completed' AND category='Technology' AND region IN ('North','South'); Inspect returned rows against all three conditions.

8. What you should see / be able to say

Every returned transfer row satisfies status <> Cancelled, region in East/West and sales >= 25000. Inspect at least three rows and separately count qualifying rows.

9. Why it worked

The query shape matches the question, the returned rows have an explicit grain, and an independent count/value/logic check can confirm the answer. SQL syntax is only useful when those three agree.

10. Common beginner mistakes

Missing parentheses when mixing AND/OR; using NOT/!= without considering NULL; filtering on text with the wrong stored spelling; writing a rule before stating it in plain language.

11. If you get stuck - recovery ladder

Unexpected rows -> translate the WHERE clause back into plain language and check AND/OR parentheses. Missing rows -> check inclusive boundaries and quoted text. NULL-related issue -> use IS NULL/IS NOT NULL, not = NULL. 1) Restate the intended result grain/question. 2) Identify the relevant clause/function. 3) Reveal only a clause skeleton, not the final query. 4) Open the matching review only after a saved attempt. 5) Close review and solve a changed retry.

SF3 - Do alone, validate and master

12. Now you do a small version alone

Return non-cancelled orders from East or West whose sales_amount is at least 25000. Use explicit parentheses so the logic cannot be misread. Then write the business rule as a comment directly above the SQL.

13. Validate your result

Every returned transfer row satisfies status <> Cancelled, region in East/West and sales >= 25000. Inspect at least three rows and separately count qualifying rows.

14. Teach it back in your own words

Explain WHERE, BETWEEN, IN, LIKE and the need for parentheses with mixed AND/OR. State one validation method for an important filter.

15. Protected review + changed fresh retry

Save the SF3 attempt in the Evidence Log first. If needed, open the protected SF3 review, diagnose only the weak idea, close the review, then solve a changed query from a blank query tab. Append the retry; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, a result screenshot/text export if required, at least one validation query/check, one error/recovery note, a one-line result-grain statement, and a concise business interpretation. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh changed retry. FAIL -> return only to this lesson's weak competency; do not restart the full SQL module.

SF4 - ORDER BY and LIMIT

Practice block: SF4 in C1_M03_SQL_FOUNDATIONS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

A SQL query can run and still answer the wrong question. This lesson builds one reliable piece of the single-table query pipeline and connects syntax to result grain and validation.

2. What you are learning

Sort results deterministically, use multiple sort keys for ties, and use ORDER BY + LIMIT together for top-N questions.

3. What you already need / prerequisite

Prerequisite: SF3. The new clause/idea is still taught from first principles.

4. Picture it in your head

ORDER BY defines result order; LIMIT only cuts the result after ordering. LIMIT 5 without ORDER BY means 'give me five rows', not 'top five'. Multiple sort keys define tie-breaking from left to right.

5. New words before we use them

ORDER BY = sort returned rows. ASC = ascending. DESC = descending. LIMIT = cap returned row count. Top-N = highest/lowest rows only when ordering is explicit.

6. Watch / visual source

RECOMMENDED. Learn SQL Beginner to Advanced in Under 4 Hours - Alex The Analyst | 47:41-51:09. Why watch: Use Alex for LIMIT/aliasing orientation; the learner book fully teaches ORDER BY. The preserved Codebasics companion covers ORDER BY/LIMIT in the broader beginner segment. Internal fallback: The learner book includes the complete SQL pattern, expected result logic and recovery path.

[Open mapped source](#)

SF4 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

SELECT order_id, sales_amount FROM orders ORDER BY sales_amount DESC LIMIT 5; Remove LIMIT temporarily to confirm the ordering continues downward.

8. What you should see / be able to say

The first 7 rows are the smallest eligible sales values under the specified tie rule; the no-LIMIT version confirms row 8 is not smaller than row 7 under the sort definition.

9. Why it worked

The query shape matches the question, the returned rows have an explicit grain, and an independent count/value/logic check can confirm the answer. SQL syntax is only useful when those three agree.

10. Common beginner mistakes

Using LIMIT without ORDER BY and calling it top-N; sorting text instead of the intended numeric/date field; forgetting ASC/DESC; treating one arbitrary returned row as meaningful.

11. If you get stuck - recovery ladder

Wrong top-N -> inspect ORDER BY direction and whether LIMIT is applied after the correct filter. Ties appear random -> add an explicit secondary sort key. 1) Restate the intended result grain/question. 2) Identify the relevant clause/function. 3) Reveal only a clause skeleton, not the final query. 4) Open the matching review only after a saved attempt. 5) Close review and solve a changed retry.

SF4 - Do alone, validate and master

12. Now you do a small version alone

Return the 7 smallest Completed orders by sales_amount; for equal sales, sort newer order_date first. Remove LIMIT once to verify the boundary between row 7 and row 8.

13. Validate your result

The first 7 rows are the smallest eligible sales values under the specified tie rule; the no-LIMIT version confirms row 8 is not smaller than row 7 under the sort definition.

14. Teach it back in your own words

Explain why ORDER BY defines what 'top' means and LIMIT only restricts how many already-ordered rows you keep.

15. Protected review + changed fresh retry

Save the SF4 attempt in the Evidence Log first. If needed, open the protected SF4 review, diagnose only the weak idea, close the review, then solve a changed query from a blank query tab. Append the retry; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, a result screenshot/text export if required, at least one validation query/check, one error/recovery note, a one-line result-grain statement, and a concise business interpretation. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh changed retry. FAIL -> return only to this lesson's weak competency; do not restart the full SQL module.

SF5 - Aggregate functions

Practice block: SF5 in C1_M03_SQL_FOUNDATIONS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

A SQL query can run and still answer the wrong question. This lesson builds one reliable piece of the single-table query pipeline and connects syntax to result grain and validation.

2. What you are learning

Choose COUNT, SUM, AVG, MIN or MAX for a question and explain exactly which filtered rows the aggregate summarizes, including COUNT(*) vs COUNT(column) with NULLs.

3. What you already need / prerequisite

Prerequisite: SF4. The new clause/idea is still taught from first principles.

4. Picture it in your head

An aggregate compresses many eligible rows into one summary value unless grouping is added later. WHERE decides which rows reach the aggregate. COUNT(*) counts rows; COUNT(column) counts non-NULL values in that column.

5. New words before we use them

Aggregate = many rows summarized into a value. COUNT = count. SUM = total. AVG = mean. MIN/MAX = smallest/largest. COUNT(*) and COUNT(column) differ when column contains NULL.

6. Watch / visual source

RECOMMENDED. Learn SQL Beginner to Advanced in Under 4 Hours - Alex The Analyst | 33:54-44:18. Why watch: See the exact SQL clause/interface behavior before typing the matching query yourself. Internal fallback: The learner book includes the complete SQL pattern, expected result logic and recovery path.

[Open mapped source](#)

SF5 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

SELECT COUNT(*) AS completed_orders, SUM(sales_amount) AS completed_sales, AVG(delivery_days) AS avg_delivery_days FROM orders WHERE status='Completed'; The locked lab should give 30 completed orders and completed sales = 691500.

8. What you should see / be able to say

Completed checkpoint remains 30 rows and 691500 sales. Transfer pending_count matches the independent count. Explain whether AVG(sales_amount) averages orders, customers or months - it averages the eligible rows.

9. Why it worked

The query shape matches the question, the returned rows have an explicit grain, and an independent count/value/logic check can confirm the answer. SQL syntax is only useful when those three agree.

10. Common beginner mistakes

Confusing COUNT(*) with COUNT(nullable_column); averaging a field without considering missing values; mixing row count and business total; failing to sanity-check min/max.

11. If you get stuck - recovery ladder

Wrong aggregate -> restate the business question and eligible row set. Suspicious average -> check filter and source grain. COUNT mismatch -> check whether you used COUNT(*) or a nullable column. 1) Restate the intended result grain/question. 2) Identify the relevant clause/function. 3) Reveal only a clause skeleton, not the final query. 4) Open the matching review only after a saved attempt. 5) Close review and solve a changed retry.

SF5 - Do alone, validate and master

12. Now you do a small version alone

For Pending orders only, return pending_count, pending_sales, average sales and maximum sales in one row. Then independently count Pending rows with a separate query.

13. Validate your result

Completed checkpoint remains 30 rows and 691500 sales. Transfer pending_count matches the independent count. Explain whether AVG(sales_amount) averages orders, customers or months - it averages the eligible rows.

14. Teach it back in your own words

Explain COUNT(*), COUNT(column), SUM, AVG, MIN and MAX using the idea of an eligible row set.

15. Protected review + changed fresh retry

Save the SF5 attempt in the Evidence Log first. If needed, open the protected SF5 review, diagnose only the weak idea, close the review, then solve a changed query from a blank query tab. Append the retry; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, a result screenshot/text export if required, at least one validation query/check, one error/recovery note, a one-line result-grain statement, and a concise business interpretation. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh changed retry. FAIL -> return only to this lesson's weak competency; do not restart the full SQL module.

SF6 - GROUP BY and result grain

Practice block: SF6 in C1_M03_SQL_FOUNDATIONS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

A SQL query can run and still answer the wrong question. This lesson builds one reliable piece of the single-table query pipeline and connects syntax to result grain and validation.

2. What you are learning

Group rows by one or more business dimensions, state result grain before running, and reconcile grouped counts back to the filtered source when groups are complete/mutually exclusive.

3. What you already need / prerequisite

Prerequisite: SF5. The new clause/idea is still taught from first principles.

4. Picture it in your head

GROUP BY changes what one result row represents. Source orders grain is one row per order; GROUP BY region changes result grain to one row per region. GROUP BY region, status changes grain to one row per region + status combination.

5. New words before we use them

GROUP BY = form buckets before aggregation. Group key = field(s) defining each bucket. Grouped result grain = one row per unique group-key combination.

6. Watch / visual source

RECOMMENDED. Learn SQL Beginner to Advanced in Under 4 Hours - Alex The Analyst | 33:54-44:17. Why watch: See the exact SQL clause/interface behavior before typing the matching query yourself. Internal fallback: The learner book includes the complete SQL pattern, expected result logic and recovery path.

[Open mapped source](#)

SF6 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

SELECT category, COUNT(*) AS orders, AVG(delivery_days) AS avg_delivery_days FROM orders GROUP BY category ORDER BY avg_delivery_days DESC; State the result grain before interpreting 'slowest'.

8. What you should see / be able to say

The transfer result grain is one row per channel among Completed orders. Sum of completed_count across channels equals 30 on the locked course data.

9. Why it worked

The query shape matches the question, the returned rows have an explicit grain, and an independent count/value/logic check can confirm the answer. SQL syntax is only useful when those three agree.

10. Common beginner mistakes

Selecting a non-aggregated column that is not part of GROUP BY; forgetting that the result grain changes to one row per group; trusting totals without reconciling to the ungrouped source.

11. If you get stuck - recovery ladder

SQL error about nonaggregated columns -> check that every selected detail column is grouped or aggregated. Wrong grain -> remove/add grouping columns deliberately. Count mismatch -> check filter consistency and whether groups overlap. 1) Restate the intended result grain/question. 2) Identify the relevant clause/function. 3) Reveal only a clause skeleton, not the final query. 4) Open the matching review only after a saved attempt. 5) Close review and solve a changed retry.

SF6 - Do alone, validate and master

12. Now you do a small version alone

For Completed orders only, return one row per channel with completed_count and completed_sales. Reconcile the sum of group counts to the independent Completed-order count.

13. Validate your result

The transfer result grain is one row per channel among Completed orders. Sum of completed_count across channels equals 30 on the locked course data.

14. Teach it back in your own words

Explain how GROUP BY changes result grain. Give one example of a one-column group and a two-column group, plus one reconciliation check.

15. Protected review + changed fresh retry

Save the SF6 attempt in the Evidence Log first. If needed, open the protected SF6 review, diagnose only the weak idea, close the review, then solve a changed query from a blank query tab. Append the retry; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, a result screenshot/text export if required, at least one validation query/check, one error/recovery note, a one-line result-grain statement, and a concise business interpretation. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh changed retry. FAIL -> return only to this lesson's weak competency; do not restart the full SQL module.

SF7 - HAVING vs WHERE

Practice block: SF7 in C1_M03_SQL_FOUNDATIONS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

A SQL query can run and still answer the wrong question. This lesson builds one reliable piece of the single-table query pipeline and connects syntax to result grain and validation.

2. What you are learning

Decide whether a condition applies to source rows or grouped results, use WHERE before grouping and HAVING for aggregate/group conditions, and explain the query in business language.

3. What you already need / prerequisite

Prerequisite: SF6. The new clause/idea is still taught from first principles.

4. Picture it in your head

WHERE filters source rows before GROUP BY. HAVING filters groups after aggregate values exist. 'Only Completed orders' is a row rule; 'regions with at least 10 Completed orders' is a group rule.

5. New words before we use them

HAVING = filter groups after aggregation. WHERE = filter source rows before grouping. Aggregate condition = condition such as `SUM(sales_amount) > 100000`.

6. Watch / visual source

RECOMMENDED. Learn SQL Beginner to Advanced in Under 4 Hours - Alex The Analyst | 44:18-47:40. Why watch: See the exact SQL clause/interface behavior before typing the matching query yourself. Internal fallback: The learner book includes the complete SQL pattern, expected result logic and recovery path.

[Open mapped source](#)

SF7 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

SELECT channel, COUNT(*) AS completed_orders FROM orders WHERE status='Completed' GROUP BY channel HAVING COUNT(*) >= 8; Explain which condition acts before grouping and which acts after.

8. What you should see / be able to say

All source rows entering the transfer groups are Completed. Every surviving region meets the aggregate threshold. A version without HAVING shows the groups that were filtered out.

9. Why it worked

The query shape matches the question, the returned rows have an explicit grain, and an independent count/value/logic check can confirm the answer. SQL syntax is only useful when those three agree.

10. Common beginner mistakes

Using HAVING for ordinary row filtering; placing aggregate conditions in WHERE; not distinguishing rows-before-grouping from groups-after-aggregation.

11. If you get stuck - recovery ladder

Aggregate condition in WHERE -> move it to HAVING. Row-level condition in HAVING -> move it to WHERE unless there is a deliberate reason. If result is empty, keep the correct query and explain what the data means. 1) Restate the intended result grain/question. 2) Identify the relevant clause/function. 3) Reveal only a clause skeleton, not the final query. 4) Open the matching review only after a saved attempt. 5) Close review and solve a changed retry.

SF7 - Do alone, validate and master

12. Now you do a small version alone

Using only Completed orders, show regions whose completed_sales is at least 150000. Include completed_count and completed_sales. State the result grain and explain the WHERE/HAVING split.

13. Validate your result

All source rows entering the transfer groups are Completed. Every surviving region meets the aggregate threshold. A version without HAVING shows the groups that were filtered out.

14. Teach it back in your own words

Explain WHERE vs HAVING without saying only 'HAVING comes later'. Use the row-rule versus group-rule distinction.

15. Protected review + changed fresh retry

Save the SF7 attempt in the Evidence Log first. If needed, open the protected SF7 review, diagnose only the weak idea, close the review, then solve a changed query from a blank query tab. Append the retry; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, a result screenshot/text export if required, at least one validation query/check, one error/recovery note, a one-line result-grain statement, and a concise business interpretation. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh changed retry. FAIL -> return only to this lesson's weak competency; do not restart the full SQL module.

SF8 - CASE for business classification

Practice block: SF8 in C1_M03_SQL_FOUNDATIONS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

A SQL query can run and still answer the wrong question. This lesson builds one reliable piece of the single-table query pipeline and connects syntax to result grain and validation.

2. What you are learning

Translate a documented classification rule into CASE, order threshold conditions correctly, test boundary values, and summarize the derived categories.

3. What you already need / prerequisite

Prerequisite: SF7. The new clause/idea is still taught from first principles.

4. Picture it in your head

CASE evaluates WHEN conditions top to bottom and returns the first match. Threshold bands therefore need deliberate order and explicit boundaries. A syntactically correct CASE can still be business-wrong if thresholds were invented or boundary logic is wrong.

5. New words before we use them

CASE = conditional expression returning a value. WHEN = tested condition. THEN = returned value. ELSE = fallback. Branch order matters because the first matching WHEN wins.

6. Watch / visual source

RECOMMENDED. Learn SQL Beginner to Advanced in Under 4 Hours - Alex The Analyst | 1:26:39-1:35:15. Why watch: See the exact SQL clause/interface behavior before typing the matching query yourself. Internal fallback: The learner book includes the complete SQL pattern, expected result logic and recovery path.

[Open mapped source](#)

SF8 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

Classify sales_amount >=30000 as Large, >=18000 as Medium, else Small. Separately classify delivery_days <=3 Fast, <=5 Standard, else Slow. Inspect exact boundary values 3, 5, 18000 and 30000.

8. What you should see / be able to say

Transfer band counts sum to 48. NULL rows land only in Unrated. Boundary scores 3 and 4 land in the intended bands. No row is silently excluded.

9. Why it worked

The query shape matches the question, the returned rows have an explicit grain, and an independent count/value/logic check can confirm the answer. SQL syntax is only useful when those three agree.

10. Common beginner mistakes

Putting broad conditions before narrow ones so later CASE branches never run; missing ELSE; treating CASE output as proof that source values are valid; not testing boundary values.

11. If you get stuck - recovery ladder

Wrong bands -> inspect WHEN order and boundary operators. NULL falls into ELSE -> test IS NULL explicitly before numeric comparisons. Business-rule uncertainty -> document/confirm thresholds rather than inventing them silently. 1) Restate the intended result grain/question. 2) Identify the relevant clause/function. 3) Reveal only a clause skeleton, not the final query. 4) Open the matching review only after a saved attempt. 5) Close review and solve a changed retry.

SF8 - Do alone, validate and master

12. Now you do a small version alone

Create service_band from satisfaction_score: NULL -> 'Unrated', >=4 -> 'Strong', =3 -> 'Neutral', else 'Weak'. Summarize counts by band and reconcile them to total orders.

13. Validate your result

Transfer band counts sum to 48. NULL rows land only in Unrated. Boundary scores 3 and 4 land in the intended bands. No row is silently excluded.

14. Teach it back in your own words

Explain CASE evaluation order, threshold boundaries and why classification rules need documented business meaning.

15. Protected review + changed fresh retry

Save the SF8 attempt in the Evidence Log first. If needed, open the protected SF8 review, diagnose only the weak idea, close the review, then solve a changed query from a blank query tab. Append the retry; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, a result screenshot/text export if required, at least one validation query/check, one error/recovery note, a one-line result-grain statement, and a concise business interpretation. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh changed retry. FAIL -> return only to this lesson's weak competency; do not restart the full SQL module.

End-of-module mastery checklist

- SF0: Connect, run controlled setup, verify row counts and distinguish stored data from result grid.
- SF1: State table grain, primary/foreign keys and use IS NULL / IS NOT NULL correctly.
- SF2: SELECT requested columns, alias clearly and explain DISTINCT result grain.
- SF3: Translate row-level business rules into correct WHERE logic with deliberate AND/OR parentheses.
- SF4: Use ORDER BY + LIMIT for deterministic top/bottom-N and explain why LIMIT alone is not ranking.
- SF5: Use COUNT/SUM/AVG/MIN/MAX and explain COUNT(*) vs COUNT(nullable_column).
- SF6: GROUP BY at a stated grouped result grain and reconcile group totals/counts.
- SF7: Use WHERE before grouping and HAVING after grouping for aggregate conditions.
- SF8: Build CASE classifications with correct branch order, ELSE and boundary tests.

Gate readiness

Attempt Gate A only when you can type each core pattern from a blank tab and validate it.

Clean Gate A pass ends M03. B/C are remediation retests only.

Revision / SQL quick reference

SELECT	Which columns/calculations should the result show?
FROM	Which table am I reading?
WHERE	Which source rows survive before grouping?
GROUP BY	Which rows belong in the same summary bucket?
HAVING	Which groups survive after aggregation?
ORDER BY	How should returned rows be sorted?
LIMIT	How many sorted rows should be returned?
NULL	Use IS NULL / IS NOT NULL; missing is not zero.
COUNT(*)	Counts rows; COUNT(column) skips NULL in that column.
DISTINCT	Unique combination of exactly the selected columns.
CASE	First matching WHEN returns the branch value.

Next route: Gate A -> M04 SQL for Analysts only after SQL Foundations competency pass.